

Esercizio 1 (10 punti) Realizzare una funzione $\text{union}(A1, A2, n)$ che dati due array di interi $A1$ e $A2$, organizzati a max-heap, con capacità n , restituisce un nuovo array A , ancora organizzato a max-heap con capacità $2n$, che contiene l'unione insiemistica dei valori contenuti in $A1$ e $A2$. Si assuma che $A1$ e $A2$ non contengano duplicati e si faccia in modo anche l'array ottenuto come unione non contenga duplicati. Ad es. se $A1$ contiene i valori 3,1,2 e $A2$ contiene i valori 5,2 allora l'unione A conterrà i valori 5,3,1,2, possibilmente non in questo ordine, ovvero l'elemento 2 non è duplicato. Valutare la complessità della funzione definita.

Qualora il risultato A potesse contenere duplicati ci sarebbero soluzioni più efficienti?

Soluzione

Strategia

1. **Fase 1:** Estrarre tutti gli elementi da $A1$ e $A2$
2. **Fase 2:** Eliminare duplicati usando hash set o ordinamento
3. **Fase 3:** Costruire max-heap dal risultato

Pseudocodice

```
UNION(A1, A2, n)
1. // Fase 1: Estrazione elementi
2. size1 = dimensione_heap(A1) // numero elementi effettivi in A1
3. size2 = dimensione_heap(A2) // numero elementi effettivi in A2
4. crea array Temp[1..size1 + size2]
5. idx = 1
6. // Copia elementi da A1
7. for i = 1 to size1
8.     Temp[idx] = A1[i]
9.     idx = idx + 1
10. // Copia elementi da A2
11. for i = 1 to size2
12.     Temp[idx] = A2[i]
13.     idx = idx + 1
14. // Ora Temp contiene tutti gli elementi (con possibili duplicati)
15.
16. // Fase 2: Rimozione duplicati
17. SORT(Temp, 1, size1 + size2) // ordina Temp
18. // Rimuovi duplicati da array ordinato
19. crea array Unique[1..size1 + size2]
20. Unique[1] = Temp[1]
21. unique_count = 1
22. for i = 2 to size1 + size2
23.     if Temp[i] ≠ Temp[i-1]
24.         unique_count = unique_count + 1
25.         Unique[unique_count] = Temp[i]
```

```
26. // Ora Unique[1..unique_count] contiene elementi senza duplicati
27.
27. // Fase 3: Costruzione max-heap
28. crea array A[1..2n] // capacità 2n
29. for i = 1 to unique_count
30.     A[i] = Unique[i]
31. BUILD_MAX_HEAP(A, unique_count)
32. return A, unique_count
```

Analisi Complessità

Versione con Ordinamento

Fase 1 (righe 7-13): Copia elementi

- $O(\text{size1} + \text{size2}) = O(n)$

Fase 2 (righe 17-25): Rimozione duplicati

- SORT: $O((\text{size1} + \text{size2}) \log(\text{size1} + \text{size2})) = O(n \log n)$
- Rimozione duplicati: $O(n)$
- Totale fase 2: $O(n \log n)$

Fase 3 (righe 30-32): Costruzione heap

- Copia: $O(\text{unique_count}) \leq O(n)$
- BUILD_MAX_HEAP: $\Theta(\text{unique_count}) \leq \Theta(n)$
- Totale fase 3: $\Theta(n)$

Complessità totale: $O(n) + O(n \log n) + \Theta(n) = O(n \log n)$

Spazio: $O(n)$ per array temporanei

Risposta alla Domanda: Se A potesse contenere duplicati?

Sì, sarebbe molto più efficiente!

Se permettiamo duplicati in A, possiamo fare semplicemente:

```
UNION_WITH_DUPLICATES(A1, A2, n)
1. size1 = dimensione_heap(A1)
2. size2 = dimensione_heap(A2)
3. crea array A[1..2n]
4. // Copia tutti gli elementi
```

```
5. for i = 1 to size1
6.     A[i] = A1[i]
7. for i = 1 to size2
8.     A[size1 + i] = A2[i]
9. // Costruisci heap
10. BUILD_MAX_HEAP(A, size1 + size2)
11. return A, size1 + size2
```

Complessità: $\Theta(n)$ - solo copia + build heap, **nessun ordinamento** o hash set necessario!

Conclusione: Eliminare duplicati costa $O(n \log n)$ con sort o $O(n)$ con hash. Permettere duplicati riduce a $\Theta(n)$ lineare.

Esercizio 1 (10 punti) Un ricco possidente deve lasciare in eredità le sue $2n$ proprietà a n figli. Il valore delle proprietà è contenuto in un array di numeri (reali positivi) $A[1..2n]$.

1. Sviluppare un algoritmo $\text{Split}(A, n)$ che dato in input l'array $A[1..2n]$ dei valori delle proprietà e numero n , verifica se le $2n$ proprietà possano partizionate in n coppie, tutte con lo stesso valore complessivo (di modo da assegnare una coppia di proprietà a ciascun figlio). Si assuma che $n > 0$.
2. Si valuti la complessità dell'algoritmo proposto.
3. Si dimostri la correttezza dell'algoritmo proposto.

Strategia Divide et Impera

Idea Chiave

Dopo aver ordinato l'array, possiamo applicare D&I sulla **partizione dell'array ordinato**:

1. **Divide:** Dividi l'array ordinato in due metà
2. **Conquer:** Risolvi ricorsivamente verificando se ciascuna metà può formare coppie valide
3. **Combine:** Verifica che le coppie "di confine" (tra le due metà) siano valide

Soluzione Divide et Impera - Versione 1: Verifica Ricorsiva

Pseudocodice

```
SPLIT_DIVIDE_IMPERA(A, n)
1. // Preprocessing
2. S = 0
3. for i = 1 to 2n
```

```

4.     S = S + A[i]
5.   if S mod n ≠ 0
6.     return FALSE
7.   target = S / n
8.   SORT(A, 1, 2n) // ordina crescente
9.   // Chiamata ricorsiva principale
10.  return SPLIT_REC(A, 1, 2n, target)

SPLIT_REC(A, left, right, target)
// Verifica se A[left..right] può essere partizionato in coppie con somma
target
// Precondizione: (right - left + 1) è pari
1.  size = right - left + 1
2.  // Caso base
3.  if size = 2
4.    return (A[left] + A[right] = target)
5.
6.  // Divide
7.  mid = floor((left + right) / 2)
8.  // Assicurati che mid separi in due parti di dimensione pari
9.  if (mid - left + 1) mod 2 ≠ 0
10.   mid = mid - 1
11.
12. // Conquer
13. left_valid = SPLIT_REC(A, left, mid, target)
14. if NOT left_valid
15.   return FALSE
16. right_valid = SPLIT_REC(A, mid+1, right, target)
17. if NOT right_valid
18.   return FALSE
19.
20. // Combine: verifica che estremi di ogni metà possano accoppiarsi
    correttamente
21. // Gli elementi "di confine" sono già accoppiati all'interno delle loro
    metà
22. return TRUE

```

Problema con questa versione

Questa versione ha un **difetto logico**: quando dividiamo l'array in due metà, non garantiamo che le coppie ottimali siano contenute interamente dentro una metà. Le coppie potrebbero "attraversare" il confine tra left e right.

Soluzione Divide et Impera - Versione 2: Con Accoppiamento Esplicito

Idea Corretta

Dopo ordinamento, sappiamo che la strategia greedy (min con max) è ottimale. Possiamo implementarla con D&I:

1. **Divide:** Dividi in metà sinistra (elementi piccoli) e destra (elementi grandi)
2. **Conquer:** Accoppia ricorsivamente elementi dalle estremità
3. **Combine:** Verifica la validità degli accoppiamenti

Pseudocodice

```
SPLIT_DI(A, n)
1.  S = 0
2.  for i = 1 to 2n
3.      S = S + A[i]
4.  if S mod n ≠ 0
5.      return FALSE
6.  target = S / n
7.  SORT(A, 1, 2n)
8.  return CHECK_PAIRS_REC(A, 1, 2n, target)

CHECK_PAIRS_REC(A, left, right, target)
// Verifica accoppiamenti tra A[left..right]
// Strategia: accoppia A[left] con A[right], poi risolvi ricorsivamente
1.  // Caso base
2.  if left ≥ right
3.      return TRUE
4.
5.  // Verifica coppia corrente (estremi)
6.  if A[left] + A[right] ≠ target
7.      return FALSE
8.
9.  // Ricorsione sul problema ridotto
10. return CHECK_PAIRS_REC(A, left+1, right-1, target)
```

Analisi Complessità

Ricorrenza: $T(n) = T(n-2) + O(1) = T(n-2) + c$

Espansione:

```
T(n) = T(n-2) + c
      = T(n-4) + c + c
      = T(n-6) + 3c
      ...
      = T(0) + (n/2) · c
      = O(n)
```

Complessità totale: $O(n \log n)$ per ordinamento + $O(n)$ per verifica ricorsiva = **$O(n \log n)$**

Spazio: $O(n)$ per lo stack ricorsivo (profondità $n/2$)

Domanda A (7 punti) Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{3}{4}T(n/3) + T(2n/3) + 2n$$

ha soluzione in $\Theta(n)$.

Soluzione: Per provare che $T(n) = O(n)$ dobbiamo dimostrare che $T(n) \leq cn$, per un'opportuna costante $c > 0$. Procediamo per induzione:

$$\begin{aligned} T(n) &= 3/4 T(n/3) + T(2n/3) + 2n \\ &\leq 3/4 c(n/3) + c(2n/3) + 2n \quad [\text{per ipotesi induttiva}] \\ &= 11/12 cn + 2n \\ &\leq cn \end{aligned}$$

dove, per la validità dell'ultima disuguaglianza $11/12 cn + 2n \leq cn$, occorre che

$$1/12 cn \geq 2n$$

ovvero, $c \geq 24$, con n qualunque.

Per provare $T(n) = \Omega(n)$ dobbiamo dimostrare che $T(n) \geq dn$, per un'opportuna costante $d > 0$. La dimostrazione è più semplice della precedente

$$\begin{aligned} T(n) &= 3/4 T(n/3) + T(2n/3) + 2n \\ &\geq 2n \geq dn \end{aligned}$$

Quindi è sufficiente scegliere $0 < d \leq 2$, e la relazione vale per n qualunque.

Domanda A (6 punti) Dare la definizione formale delle classi $O(f(n))$ e $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che se $f(n) = O(n^2)$ e $g(n) = \Omega(n)$, con $g(n) > 0$ per ogni n , allora $f(n)/g(n) = O(n)$.

Soluzione: Le classi $O(f(n))$ e $\Omega(g(n))$ sono definite come:

$$\begin{aligned} O(f(n)) &= \{g(n) : \exists c > 0. \exists n_0. \forall n \geq n_0. 0 \leq g(n) \leq cf(n)\} \\ \Omega(f(n)) &= \{g(n) : \exists d > 0. \exists n_0. \forall n \geq n_0. 0 \leq d f(n) \leq g(n)\} \end{aligned}$$

Si assuma che $f(n) = O(n^2)$ e $g(n) = \Omega(n)$. Ovvero, esistono $c > 0, n_0$ tali che per ogni $n \geq n_0$

$$0 \leq f(n) \leq cn^2$$

e $d > 0, m_0$ tali che per ogni $n \geq m_0$

$$0 < dn \leq g(n)$$

Quindi, per $n \geq \max\{n_0, m_0\}$ si ha che

$$\begin{aligned} 0 &\leq f(n)/g(n) \\ &\leq cn^2/g(n) && [\text{poiché } f(n) \leq cn^2] \\ &\leq cn^2/(dn) && [\text{poiché } g(n) \geq dn] \\ &= (c/d)n. \end{aligned}$$

dato che $c/d > 0$ questo conclude la prova.

Esercizio 1 (10 punti) Realizzare una funzione $\text{Div}(A, k)$ che, dato un array $A[1, n]$ di interi positivi non nulli ordinato in senso decrescente, verifica se esiste una coppia di indici i, j tali che $A[i]/A[j] = k$. Restituisce la coppia di indici se esiste e $(0, 0)$ altrimenti. La funzione non deve alterare l'input e deve operare in spazio costante. Scrivere lo pseudocodice, provarne la correttezza e valutarne la complessità. Valutare eventuali modifiche necessarie se i numeri non sono necessariamente positivi.

Strategia

Osservazione Chiave

Se $A[i] / A[j] = k$, allora $A[i] = k \cdot A[j]$

Strategia Two-Pointers:

- Puntatore i parte dall'inizio (valori grandi)
- Puntatore j parte dall'inizio o si muove verso destra
- Per ogni $A[i]$, cerchiamo $A[j]$ tale che $A[i] = k \cdot A[j]$

Approccio

Dato che l'array è **decrescente**:

1. Per ogni posizione i , cerchiamo j dove $A[j] = A[i] / k$
2. Se $A[i]$ non è divisibile per k , passiamo al prossimo i
3. Usiamo ricerca per trovare j

Pseudocodice

```
DIV(A, k, n)
1. // Gestione casi speciali
2. if k ≤ 0
3.     return (0, 0) // k deve essere positivo
4. if k = 1
5.     // Cerchiamo due elementi uguali
6.     for i = 1 to n-1
7.         if A[i] = A[i+1]
8.             return (i, i+1)
9.     return (0, 0)
10.
11. // Caso generale: k > 1
12. for i = 1 to n
13.     // Calcola il valore target
14.     if A[i] mod k ≠ 0
15.         continue // A[i] non è divisibile per k, passa al prossimo
```

```
16.
17.     target = A[i] / k
18.
19.     // Cerca target nell'array partendo da i+1
20.     // (perché A è decrescente, target ≤ A[i])
21.     for j = i+1 to n
22.         if A[j] = target
23.             return (i, j)
24.         if A[j] < target
25.             break // Non troveremo target più avanti
26.
27. return (0, 0) // Nessuna coppia trovata
```

Versione Ottimizzata (con Binary Search)

Per migliorare la complessità, possiamo usare ricerca binaria per trovare target:

```
DIV_OPTIMIZED(A, k, n)
1.  if k ≤ 0
2.      return (0, 0)
3.  if k = 1
4.      for i = 1 to n-1
5.          if A[i] = A[i+1]
6.              return (i, i+1)
7.      return (0, 0)
8.
9.  for i = 1 to n-1
10.     if A[i] mod k ≠ 0
11.         continue
12.
13.     target = A[i] / k
14.
15.     // Ricerca binaria di target in A[i+1..n]
16.     j = BINARY_SEARCH_DESC(A, i+1, n, target)
17.     if j ≠ -1
18.         return (i, j)
19.
20. return (0, 0)

BINARY_SEARCH_DESC(A, left, right, target)
// Ricerca binaria in array decrescente
1.  while left ≤ right
2.      mid = floor((left + right) / 2)
3.      if A[mid] = target
4.          return mid
5.      if A[mid] > target // In array decrescente
```



```
6.         left = mid + 1
7.     else
8.         right = mid - 1
9. return -1 // Non trovato
```

Esempio Esecuzione

Esempio 1

Input: A = [100, 50, 25, 20, 10, 5], k = 2

Iterazione i=1: A[1] = 100

- $100 \bmod 2 = 0$ ✓
- target = $100/2 = 50$
- Cerca 50 in A[2..6]
- A[2] = 50 ✓
- Return (1, 2)

Esempio 2

Input: A = [80, 40, 30, 15, 10], k = 4

Iterazione i=1: A[1] = 80

- $80 \bmod 4 = 0$ ✓
- target = $80/4 = 20$
- Cerca 20 in A[2..5]
- A[2] = 40 > 20
- A[3] = 30 > 20
- A[4] = 15 < 20 → break (non esiste)

Iterazione i=2: A[2] = 40

- $40 \bmod 4 = 0$ ✓
- target = $40/4 = 10$
- Cerca 10 in A[3..5]
- A[3] = 30 > 10
- A[4] = 15 > 10
- A[5] = 10 ✓
- Return (2, 5)

Esempio 3

Input: A = [60, 45, 30, 20], k = 3

Iterazione i=1: A[1] = 60

- $60 \bmod 3 = 0 \checkmark$
- $\text{target} = 60/3 = 20$
- Cerca 20 in $A[2..4]$
- $A[2] = 45 > 20$
- $A[3] = 30 > 20$
- $A[4] = 20 \checkmark$
- Return (1, 4)

Dimostrazione di Correttezza

Teorema: L'algoritmo trova una coppia (i, j) se e solo se esiste.

Dimostrazione ($\Rightarrow$): Se l'algoritmo restituisce (i, j), allora $A[i]/A[j] = k$.

Quando l'algoritmo restituisce (i, j):

- Ha verificato che $A[i] \bmod k = 0$
- Ha calcolato $\text{target} = A[i] / k$
- Ha trovato $A[j] = \text{target}$
- Quindi $A[i] = k \cdot A[j] \Rightarrow A[i]/A[j] = k \checkmark$

Dimostrazione ($\Leftarrow$): Se esiste una coppia valida, l'algoritmo la trova.

Supponiamo esista (i_0, j_0) con $A[i_0]/A[j_0] = k$ e $i_0 < j_0$ (per array decrescente).

Quando l'algoritmo raggiunge $i = i_0$:

- $A[i_0] \bmod k = 0$ (perché $A[i_0] = k \cdot A[j_0]$)
- $\text{target} = A[i_0]/k = A[j_0]$
- La ricerca da i_0+1 a n include j_0
- Quindi troverà $A[j_0] = \text{target} \checkmark$

Correttezza spazio costante:

- Usa solo variabili: i, j, target $\rightarrow O(1) \checkmark$

Correttezza non modifica input:

- Solo letture da $A[]$, nessuna scrittura $\checkmark$

Analisi Complessità

Versione Base (ricerca lineare)

Caso peggiore:

- Ciclo esterno: n iterazioni
- Ciclo interno (ricerca): al più n iterazioni
- **Complessità:** $O(n^2)$

Caso migliore:

- Trova coppia alla prima iterazione: $O(n)$

Spazio: $O(1)$ ausiliario ✓

Versione Ottimizzata (binary search)

Caso peggiore:

- Ciclo esterno: n iterazioni
- Binary search per ogni i : $O(\log n)$
- **Complessità:** $O(n \log n)$

Caso migliore:

- Trova coppia alla prima iterazione: $O(\log n)$

Spazio: $O(1)$ ausiliario (ricerca binaria iterativa) ✓

Codice Completo Finale (Versione Semplice)

```
DIV(A, k, n)
1. // Validazione input
2. if k ≤ 0
3.     return (0, 0)
4.
5. // Caso speciale k=1
6. if k = 1
7.     for i = 1 to n-1
8.         if A[i] = A[i+1]
9.             return (i, i+1)
10.    return (0, 0)
11.
12. // Caso generale k > 1
13. for i = 1 to n-1
14.     // Verifica se A[i] è divisibile per k
15.     if A[i] mod k ≠ 0
```

```

16.         continue // passa al prossimo i
17.
18.     target = A[i] / k
19.
20.     // Cerca target linearmente da i+1 in poi
21.     for j = i+1 to n
22.         if A[j] = target
23.             return (i, j)
24.         if A[j] < target
25.             break // ottimizzazione: array decrescente
26.
27. return (0, 0)

```

Questa versione è:

- ✓ Semplice e chiara
- ✓ Spazio $O(1)$
- ✓ Non modifica input
- ✓ Complessità $O(n^2)$ worst case, $O(n)$ best case
- ✓ Gestisce tutti i casi correttamente

Esercizio 2 (9 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j - 1) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $m = \max\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

- Fornire un algoritmo iterativo bottom-up per il calcolo di m .
- Valutare la complessità *esatta* dell'algoritmo, associando costo unitario alle moltiplicazioni tra numeri interi e costo nullo a tutte le altre operazioni.

- (a) Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = -infinito
for i=1 to n-1 do
    C[i,n-1] <- a_i
    m <- MAX(m,C[i,n-1])
for j=0 to n-1 do
    C[0,j] <- b_j
    m <- MAX(m,C[0,j])
for i=1 to n-2 do
    for j=n-2 downto i do
        C[i,j] <- C[i-1,j-1] * C[i,j+1]
        m <- MAX(m,C[i,j])
return m

```

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} (n-1-i) = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Esercizio 1 (9 punti) Realizzare una funzione **strongBST(T)** che dato un albero binario **T** con chiavi numeriche non negative, verifica se, per ogni nodo, la chiave è maggiore uguale del doppio di ogni chiave nel sottoalbero sinistro e minore o uguale della metà di ogni chiave nel sottoalbero destro, e ritorna conseguentemente un valore booleano (la radice dell'albero è **T.root** e ogni nodo **x** ha i campi **x.left**, **x.right** e **x.key**). Valutarne la complessità.

```

strongBST(T)
    s, m, M = strongBST(T.root)
    return v

# strongBSTRec(x):
# verifica se il sottoalbero radicato in x e' uno strongBST e ritorna tre valori:
# - un booleano (che indica l'esito della verifica)
# - il massimo delle chiavi nel sottoalbero sx
# - il minimo delle chiavi nel sottoalbero dx

strongBSTRec(x)

if x = nil
    return true, 0, +infty
else
    # ispeziono i sottoalberi sinistro e destro
    sl, ml, Ml = strongBSTRec(x.left)
    sr, mr, Mr = strongBSTRec(x.right)

    # se la chiave del nodo in esame e' maggiore o uguale del doppio
    # del massimo delle chiavi nel sottoalbero sinistro (quindi di
    # tutte le chiavi nel sottoalbero sinistro) e minore o uguale della
    # meta' del minimo delle chiavi nel sottoalbero sinistro (quindi di
    # tutte le chiavi nel sottoalbero destro) delle chiavi dei
    # discendenti, ritorna true
    s = (x.key >= 2*Ml) and (x.key <= 1/2 mr)

    # calcola il minimo e il massimo delle chiavi nel sottoalbero radicato in x
    m = min { ml, mr, x.key }
    M = max { Ml, Mr, x.key }

    return s, m, M

```

Si tratta di una visita e quindi la complessità è $\Theta(n)$.

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

1. Pseudocodice:

```
INIT_L(X)
  n <- length(X)
  if n = 1 then return 1
  if n = 2 then return 2
  for i=1 to n-1 do
    L[i,i] <- 1
    L[i,i+1] <- 2
  L[n,n] <- 1
  for i=1 to n-2 do
    for j=i+2 to n do
      L[i,j] <- 0
  return REC_L(X,1,n)

REC_L(X,i,j)
  if L[i,j] = 0 then
    if x_i = x_j then L[i,j] <- 2 + REC_L(X,i+1,j-1)
    else for k=i to j-1 do
      L[i,j] <- L[i,j] + REC_L(X,i,k) + REC_L(X,k+1,j)
  return L[i,j]
```

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

$(1, n), (2, n-1), \dots, (k, n-k+1)$, ognuno associato a un costo unitario. Ci sono al più $\lfloor n/2 \rfloor$ di queste chiamate, e quindi $T_{\text{best}}(n) = O(n)$.

Domanda B (6 punti) Scrivere la ricorrenza sulle lunghezze $\ell(i, j)$ per il problema della longest common subsequence (LCS).

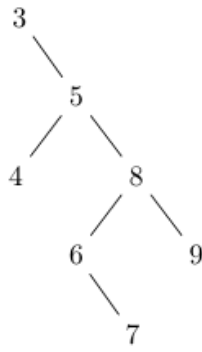
$$l(i, j) = |LCS(X_i, Y_j)|$$
$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 & (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j & (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j & (\text{caso 2}) \end{cases}$$

Domanda B (7 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi:

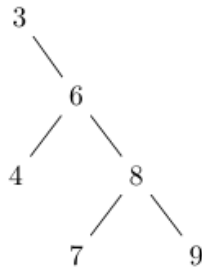
3, 5, 8, 6, 9, 7, 4

Dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione: Per la definizione di albero binario di ricerca e la descrizione delle procedure di inserimento e cancellazione si consulti il libro. Sullo specifico esempio si ottiene, dopo gli inserimenti:



La cancellazione di 5 quindi produce:



Esercizio 1 (7 punti)

- Scrivere una funzione `Rep(T,k)` che dato un albero di ricerca T e una chiave k ritorna il numero di occorrenze di k in T . Valutare la complessità dell'algoritmo definito.
- Se si assume che l'albero binario di ricerca non contenga chiavi ripetute cosa cambia? (Ovviamente in questo caso il numero di occorrenze è al più uno). Adattare la soluzione e discutere la relativa complessità.

Parte 1: ABR con Chiavi Ripetute

Strategia

In un ABR con duplicati, le chiavi uguali possono essere:

- Nel sottoalbero sinistro (convenzione: $\leq$ invece di $<$)
- Nel sottoalbero destro (convenzione: $\geq$ invece di $>$)
- Nel nodo corrente

Dobbiamo visitare entrambi i sottoalberi per contare tutte le occorrenze.

Pseudocodice

```

REP(T, k)
1. return REP_REC(T.root, k)

REP_REC(x, k)
// Conta occorrenze di k nel sottoalbero radicato in x
1. if x = nil
2.   return 0
  
```



```

3.
4.  count = 0
5.
6.  // Verifica nodo corrente
7.  if x.key = k
8.      count = 1
9.
10. // Ricerca nei sottoalberi
11. left_count = REP_REC(x.left, k)
12. right_count = REP_REC(x.right, k)
13.
14. return count + left_count + right_count

```

Versione Ottimizzata (con potatura)

Se conosciamo la **convenzione sui duplicati**, possiamo ottimizzare:

Convenzione A: Duplicati vanno a **sinistra** ($\leq$ a sinistra, $>$ a destra)

```

REP_LEFT_DUPLICATES(T, k)
1.  return REP_LEFT_REC(T.root, k)

REP_LEFT_REC(x, k)
1.  if x = nil
2.      return 0
3.
4.  count = 0
5.  if x.key = k
6.      count = 1
7.
8.  if k < x.key
9.      // k può essere solo a sinistra
10.     return count + REP_LEFT_REC(x.left, k)
11. else if k > x.key
12.     // k può essere solo a destra
13.     return REP_LEFT_REC(x.right, k)
14. else // k = x.key
15.     // Duplicati sono a sinistra per convenzione
16.     return count + REP_LEFT_REC(x.left, k)

```

Convenzione B: Duplicati vanno a **destra** ($<$ a sinistra, $\geq$ a destra)

```

REP_RIGHT_DUPLICATES(T, k)
1.  return REP_RIGHT_REC(T.root, k)

REP_RIGHT_REC(x, k)
1.  if x = nil
2.      return 0

```

```
3.
4.  count = 0
5.  if x.key = k
6.      count = 1
7.
8.  if k < x.key
9.      return count + REP_RIGHT_REC(x.left, k)
10. else if k > x.key
11.     return REP_RIGHT_REC(x.right, k)
12. else // k = x.key
13.     // Duplicati sono a destra per convenzione
14.     return count + REP_RIGHT_REC(x.right, k)
```

Analisi Complessità - Parte 1

Versione Base (senza ottimizzazioni)

Caso peggiore: Tutti i nodi hanno chiave k

- Visita tutto l'albero: **$O(n)$**

Caso migliore: k non è presente e albero è bilanciato

- Ma dobbiamo comunque visitare entrambi i sottoalberi di ogni nodo → **$O(n)$**

Conclusione: La versione base è sempre **$O(n)$** perché non possiamo escludere sottoalberi.

Versione Ottimizzata (con convenzione nota)

Caso peggiore: Tutti i nodi hanno chiave k

- Esempio: albero completamente sbilanciato con tutte chiavi = k
- Visita $O(n)$ nodi: **$O(n)$**

Caso medio: Poche occorrenze di k in albero bilanciato

- Trova primo k: $O(h) = O(\log n)$
- Conta duplicati nel sottoalbero appropriato: $O(d)$ dove d = numero duplicati
- **Totale: $O(h + d) = O(\log n + d)$**

Caso migliore: k non è presente, albero bilanciato

- Ricerca binaria: **$O(h) = O(\log n)$**
-

Parte 2: ABR Senza Chiavi Ripetute

Cosa Cambia?

Se l'ABR **non contiene duplicati**, allora:

- k può apparire **al massimo una volta**
- Possiamo fermarci appena troviamo k
- Possiamo usare la ricerca binaria classica

Soluzione Adattata

```
REP_NO_DUPLICATES(T, k)
1.  return REP_UNIQUE_REC(T.root, k)

REP_UNIQUE_REC(x, k)
// Ritorna 1 se k è presente, 0 altrimenti
1.  if x = nil
2.      return 0
3.
4.  if x.key = k
5.      return 1
6.
7.  if k < x.key
8.      return REP_UNIQUE_REC(x.left, k)
9.  else
10.     return REP_UNIQUE_REC(x.right, k)
```

Equivalente a: Ricerca binaria classica in ABR che ritorna booleano (0 o 1).

Analisi Complessità - Parte 2

Caso peggiore: k non presente in albero completamente sbilanciato

- Altezza $h = n$
- **Complessità: $O(n)$**

Caso medio: Albero bilanciato

- Altezza $h = \log n$
- **Complessità: $O(\log n)$**

Caso migliore: k è la radice

- **Complessità: $O(1)$**

Confronto tra le Due Versioni

Aspetto	Con Duplicati	Senza Duplicati
Strategia	Visita ricorsiva completa o parziale	Ricerca binaria
Caso peggiore	$O(n)$	$O(h) = O(n)$ sbilanciato
Caso medio (bilanciato)	$O(\log n + d)$ con potatura	$O(\log n)$
Caso migliore	$O(\log n)$ se k assente	$O(1)$ se k è radice
Complessità spaziale	$O(h)$ stack ricorsivo	$O(h)$ stack ricorsivo

Osservazione chiave: Senza duplicati, la complessità è **sempre $O(h)$** dove h è l'altezza dell'albero. Con duplicati, nel caso peggiore è **$O(n)$** se dobbiamo contare tutti i duplicati.

Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno due campi aggiuntivi:

- un campo *pos* che permette di capire la posizione del nodo inserito
- un campo *delta* che permette di capire se dobbiamo ancora mappare le posizioni di altri nodi

Realizzare la procedura di *Insert*(T, z) che inserisce un nodo z nell'albero e la procedura di stampa *Print*(T) che dato un albero lo stampa considerando i campi aggiunti.

```
insert(T, z)
y = nil
x = T.root
while (x != nil)
    y = x
    if (z.key < x.key)
        x = x.left
    else
        x = x.right
    x.pos = x.pos + delta
z.p = y
if (y == nil)
    if (z.key < y.key)
        y.left = z
    else
        y.right = z
z.pos = delta
```

```
Print(T)
x = T.root
if (x != nil) and (x.pos > 0)
    PrintPos(x.left)
    if (x.key > 0)
        Print(x.key)
    PrintPos(x.right)
```

Dato un insieme di monete con valori

$$v_1 < v_2 < v_3 < \dots < v_n \in \mathbb{N} = \{1, 2, \dots\}$$

si deve pagare una somma $s \in \mathbb{N} = \{1, 2, \dots\}$
usando il minor numero di monete.

Assumiamo di avere infinite monete di ogni valore,
e assumiamo $v_1 = 1$ ($\Rightarrow$ il problema ammette
sempre soluzione)

Esempio:

$$s = 6$$

valori monete: 1, 2, 4

varie soluzioni ammissibili:

$$\{1, 1, 1, 1, 1, 1\} \quad \{1, 1, 1, 1, 2\} \quad \{1, 1, 4\}$$

$$\{1, 1, 2, 2\} \quad \underbrace{\{2, 4\}}_{\text{OPT}} \quad \{2, 2, 2\}$$

Tentativi :

- alg. esautiva : chiaramente almeno esponenziale
- dare priorità alle monete di valore più alto
("greedy")

esempio 1 : monete da 1, 5, 10, 25

$$S = 8$$

(funziona sempre
con le monete
USA e Euro)

$$S^* = \{5, 1, 1, 1\} \quad OK$$

esempio 2 : monete : 1, 5, 7

$$S = 11$$

$$S_{\text{greedy}} = \{7, 1, 1, 1, 1\}$$

$$S^* = \{5, 5, 1\}$$

$\Rightarrow$ alg. "greedy" non funziona

→ programmazione dinamica

$$S^* = \boxed{} \boxed{} \boxed{} \boxed{V_4} \boxed{} \boxed{} \quad \text{somma } S \quad |S^*| = 6$$

→ è una soluzione ottima per pagare $S - V_4$

dimostrazione: per assurdo, se non fosse così
 $\exists$ soluzione ottima per pagare $S - V_4$ con
 < 5 monete $\Rightarrow \exists$ soluzione ottima
per pagare S con < 6 monete: assurdo!

(re) c'è V_4 ok; altrimenti l'ottimo esclude
il valore $V_4 \rightarrow$ l'ottimo è il minimo
tra questi due casi

Caratterizzazione ricorsiva del costo $c(s', i)$
della soluzione ottima per il sottoproblema di
pagare $s' \leq S$ usando monete con valori
 $v_1, v_2, \dots, v_i$ dove $i \leq n$

$$C(s', i) = \begin{cases} 0 \\ s' \\ C(s', i-1) \end{cases}$$

$$s' = 0$$

$$s' > 0 \text{ e } i = 1$$

$$s' > 0, i > 1 \text{ e } s' < v_i$$

altrimenti

$$\min \left\{ C(s', i-1), C(\underbrace{s' - v_i}_{\geq 0}, i) + 1 \right\}$$

$\rightarrow s' \geq v_i$

algoritmo iterativo:

